ADR-001 — Communication architecture for order processing

OrderFlow — ADR-001 final · Écrit après les mesures · 2026-11-09 · Statut : Accepted

Executive conclusion

Decision : ADOPT SYNCHRONOUS, avec ajout d'un système de nouvelles tentatives côté synchrone (décision de l'équipe du POC, 2026-11-09). La décision candidate (ADOPT EVENT-DRIVEN, 2026-10-07) est **infirmée** sur son argument principal : l'événementiel ne tient pas mieux le pic sur ce banc d'essai, et le surcoût d'écritures qu'il génère sature la base partagée en premier. Son avantage réel, la résilience face aux dépendances (H3a, H3b, H3c), est reconnu et traité en renforçant le synchrone.

Réserves : les verdicts sur le pic (H1) et la latence en charge normale (H2) restent **provisoires** (une répétition exploitable, la seconde ayant été faite sur une machine ralentie). Une répétition 3 en environnement propre est prévue.

Status

Accepted — 2026-11-09, décision de l'équipe du POC.

Version finale, écrite après les mesures. La version initiale, écrite avant toute mesure, est conservée telle quelle dans `docs/ADR-001-initial.md` ; la *Candidate decision* ci-dessous en est recopiée sans modification.

Réserves : les verdicts sur le pic (H1) et la latence en charge normale (H2) restent **provisoires** (une répétition exploitable, la seconde ayant été faite sur une machine ralentie) ; une répétition 3 en environnement propre est prévue (voir *Consequences*).

Context

Une plateforme de commerce électronique traite les commandes avec cinq services : Order, Payment, Inventory, Shipping, Notification. Pour chaque commande : création, validation du paiement, réservation du stock, demande d'expédition, confirmation au client.

L'entreprise prévoit une forte croissance du trafic, avec des pics promotionnels plusieurs fois supérieurs à la charge normale (paramètres pédagogiques : 100 cmd/s en charge normale, 500 cmd/s pendant 5 min en pic, p95 < 1 s pour la réponse initiale).

Quelle que soit l'architecture, un invariant doit tenir : **une commande ne peut être confirmée que si le paiement est autorisé et le stock réservé.**

La direction technique ne veut pas adopter une architecture événementielle « parce qu'elle est plus scalable » : elle demande des preuves.

Problème. Nous ne savions pas si le découplage par événements améliore suffisamment la tenue au pic et la tolérance aux pannes de dépendances pour compenser la complexité qu'il ajoute pour préserver l'invariant (idempotence, outbox, cohérence à terme). La PoC a été construite pour répondre à cette question (plan : `docs/POC-plan.md`).

Architectural drivers

Driver	Ce qui compte ici
Scalability	Tenir le pic de 500 cmd/s sans effondrement incontrôlé
Availability	Continuer à prendre en compte les commandes quand une dépendance est lente ou indisponible, et revenir à la normale après
Performance	p95 < 1 s jusqu'à l'acceptation en charge normale
Consistency	Invariant tenu, y compris en cas de message traité plusieurs fois
Observability	Pouvoir reconstruire le traitement d'une commande individuelle
Operational complexity	Coût d'exploitation et de compréhension du système

Options considered

1. **Synchronous workflow** — Order appelle successivement Payment → Inventory → Shipping → Notification (REST). Implémenté et mesuré (`sync/`).

2. **Event-driven workflow** — Order enregistre la commande et publie un événement ; les autres services réagissent aux événements (chorégraphie, RabbitMQ, outbox transactionnel, consommateurs idempotents). Implémenté et mesuré (`async/`).
3. **Hybrid** — Payment et Inventory en synchrone jusqu'à la confirmation ; Shipping et Notification par événements. **Non implémenté, non mesuré** (H6 restée en brouillon).

Les deux variantes mesurées partagent la même logique métier (`domain.js` identiques, contrôlés automatiquement), les mêmes ressources (1 CPU / 512 Mo par service), le même invariant appliqué par la même requête, la même charge et le même budget de nouvelles tentatives (zéro). Seul le style de communication diffère.

Candidate decision

(Recopiée de `docs/ADR-001-initial.md`, sans modification.)

ADOPT EVENT-DRIVEN — intuition de l'équipe, 2026-10-07.

Raisons :

- la file absorbe les pics de charge, là où la chaîne synchrone sature ;
- une dépendance lente ou indisponible ne bloque pas la prise en compte des commandes.

Cette décision est volontairement provisoire. Elle sera confirmée ou infirmée principalement par **H1** (scalabilité) et **H3** (résilience), en tenant compte du coût mesuré par **H4** (idempotence) et **H5** (observabilité).

Evidence collected

- **Six scénarios** exécutés sur les deux variantes : S1 (charge normale), S2 (pic jusqu'à 500 cmd/s), S3 (Payment lent), S4 (Payment arrêté), S4b (Shipping arrêté), S5 (événements ou requêtes en double, avec témoin et référence).
- **Deux répétitions** par scénario et par variante, en alternant l'ordre des projets.
- Pour chaque run, `results/<projet>/<scénario>/rep-<n>/` : validité, phases, mesures k6 (`run.json`), latences calculées en base par phase (`latency.json`), vérification des invariants I1 à I11 (`invariants.json`), traces (`traces.json` , S1). Ces fichiers sont versionnés.
- Synthèse chiffrée, scénario par scénario : `docs/resultats.md` . Interprétation, matrice de décision et graphiques : `docs/architecture-analysis.md` .

- Défauts du banc d'essai trouvés et corrigés avant les mesures (notamment un délai TCP de 40 ms par événement en async, qui aurait faussé la comparaison) : `docs/cas-de-comprehension.md`, `docs/async-mise-en-place.md`.

Experimental results

Hypothèse	Mesure clé (sync / async)	Verdict
Exigence S1 (100 cmd/s, p95 < 1 s)	Acceptation p95 : 68 et 71 ms / 77 ms (rép. 1)	Respectée par les deux (définitive pour sync, provisoire pour async)
H1 — l'async tient mieux le pic	À 500 cmd/s : 70,6 % accepté, 3,7 s / 43,8 %, 371 s (indicatif). Sync conforme jusqu'à 300 cmd/s, async jusqu'à 200. Goulot : Order en sync, **base partagée** en async	**Non concluante, tendance rejetée** (provisoire)
H2 — le sync complète plus vite en charge normale	Complétion p95 : 146 / 162 ms, écart 9,9 % < 20 %	**Rejetée** (provisoire)
H3a — Shipping arrêté	Commandes de la panne allées au bout : ≈ 5 % / 100 %	**Supportée** (définitive)
H3b — Payment arrêté	Commandes de la panne : ≈ 2 975 bloquées / 0 bloquée, 0 perdue des deux côtés	**Supportée** (définitive)
H3c — Payment lent	Abandons côté client : 87,6 et 87,2 % / 0 %	**Supportée** (définitive)
H4 — doublons	0 effet en double avec protection dans les deux ; médiane identique avec et sans doublons	**Supportée** (définitive)
H5 — traces plus difficiles en async	100 % de traces complètes des deux côtés	**Rejetée** (définitive)

L'invariant I1 n'a jamais été violé, dans aucune variante, même en saturation.

Trade-offs

	Synchrone (retenu)	Événementiel (écarté)
Ce qu'on gagne	Va plus loin au pic sur ce banc d'essai ; moins de ressources (une seule ressource sature : Order) ; architecture simple ; retour immédiat à la normale après une panne	Absorbe les pannes et lenteurs de dépendances : le client n'attend pas, aucune commande n'est abandonnée
Ce qu'on paie	Une dépendance lente ou en panne est répercutée au client (abandons, erreurs `502`) et les commandes concernées restent bloquées	Beaucoup plus d'écritures par commande (outbox, idempotence, une transaction par événement) : la base sature en premier au pic ; plus de mécanismes à maintenir ; une file qui fait attendre les commandes suivantes après une panne

Limitations

- **Un seul hôte** : k6, les services, la base, le broker et l'observabilité se partagent la machine ; au pic, k6 a lui-même saturé la machine, et aucun run async de S2 n'est exploitable.
- **Une base PostgreSQL unique partagée** par les cinq services. Elle pénalise l'async, dont le surcroît d'écritures tombe sur cette seule base. Avec une base par service (non testé), la conclusion sur la montée en charge pourrait changer.
- **Aucune nouvelle tentative** dans les deux variantes (règle d'équité). Une partie de l'avantage de l'async en résilience vient de là : le sync n'a pas été mesuré avec ses protections habituelles.
- **Un cœur par service** : on mesure la tenue à ressources égales, pas la montée en charge horizontale.
- Services simulés (latences et pannes injectées) ; Payment est un simulateur à réponse immédiate, sans webhook.
- Orchestration (sync) contre chorégraphie (async) : deux choses changent à la fois.
- **Deux répétitions seulement**, et la répétition 2 de S1 et S2 faite sur une machine ralentie : H1 et H2 restent provisoires.

- La confirmation précède l'expédition (D5), contrairement à la chaîne de l'énoncé.
- Défaut de mesure connu : en async, des horodatages peuvent être inversés de quelques millisecondes (I9, `now()`), sans effet sur les verdicts.

Final decision

ADOPT SYNCHRONOUS — décision de l'équipe du POC, 2026-11-09.

La décision candidate (ADOPT EVENT-DRIVEN) est infirmée sur son argument principal :

1. **L'événementiel ne tient pas mieux le pic.** Sur ce banc d'essai, le synchrone reste conforme plus longtemps (300 cmd/s contre 200) et accepte plus de commandes à 500 cmd/s. Le pic de l'événementiel sature la **base de données** : c'est le surcroît d'écritures qu'il génère (outbox, idempotence, une transaction par événement) qui la fait tomber en premier. La charge de la preuve était sur l'événementiel ; elle n'est pas apportée.
2. **En charge normale, les deux se valent** : aucun gain de latence ne justifie le changement.
3. **La cohérence est tenue dans les deux** : le synchrone ne perd rien sur l'invariant.
4. **L'avantage réel de l'événementiel est la résilience face aux dépendances** (H3a, H3b, H3c). Il est mesuré et reconnu. Il est obtenu ici par la mise en file, alors que le synchrone était testé **sans aucune nouvelle tentative**. La décision retient le synchrone et **lui ajoute un système de nouvelles tentatives** pour traiter cette faiblesse, plutôt que de payer le coût de l'événementiel sur l'ensemble du flux.

Consequences

1. **Mettre en place un système de nouvelles tentatives côté synchrone** (décision de l'équipe) : délais bornés et nouvelles tentatives sur les appels à Payment, Inventory, Shipping et Notification, en s'appuyant sur l'idempotence déjà en place (`Idempotency-Key`, `idempotency_keys`) pour qu'un appel rejoué ne crée pas de double effet ; envisager un coupe-circuit pour ne pas saturer un service déjà en difficulté. Les commandes laissées incomplètes (Shipping en panne) doivent pouvoir être reprises.
2. **Mesurer ce système** avant de le considérer acquis : rejouer S3, S4 et S4b sur le synchrone avec nouvelles tentatives, et vérifier qu'il réduit les abandons et les commandes bloquées sans violer l'invariant ni créer de doublons. C'est une

expérience distincte (la règle d'équité ne s'applique plus : on n'oppose plus deux styles, on améliore le style retenu).

3. **Accepter les limites restantes** tant que ce système n'est pas en place : une dépendance lente ou en panne reste répercutée au client.
4. **Lever les réserves sur H1 et H2** : répétition 3 de S1 et S2 en environnement propre (`docs/decisions-conception.md`, PC10).
5. **Conditions de réévaluation** : la décision serait à revoir si la répétition 3 renversait H1, si un test avec une base par service montrait que l'événementiel tient alors mieux le pic, ou si la résilience obtenue par les nouvelles tentatives restait insuffisante. L'hybride (option 3), non mesuré, reste la piste naturelle dans ce dernier cas.
6. **Abandon du code de la variante événementielle** comme cible de production ; il reste dans le dépôt comme base de comparaison de la PoC.